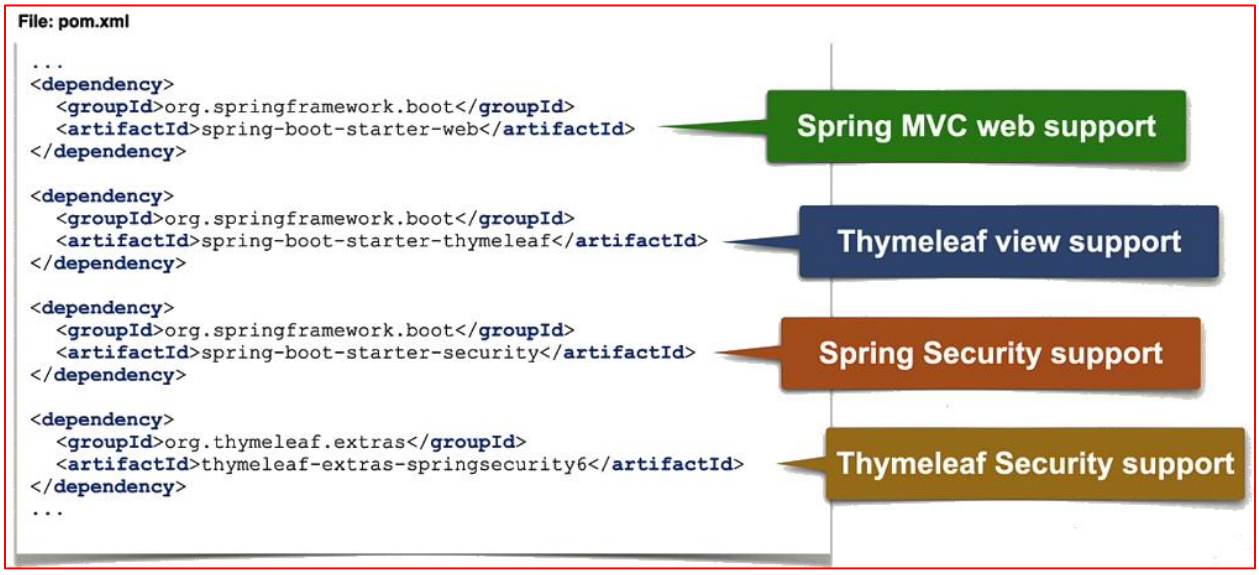
Spring MVC Security - Project Setup

In this lesson, we'll embark on the journey of setting up our Spring MVC Security Project. This lesson will guide you through a comprehensive development process, and here's how we'll proceed:

Step 1: Creating the Project

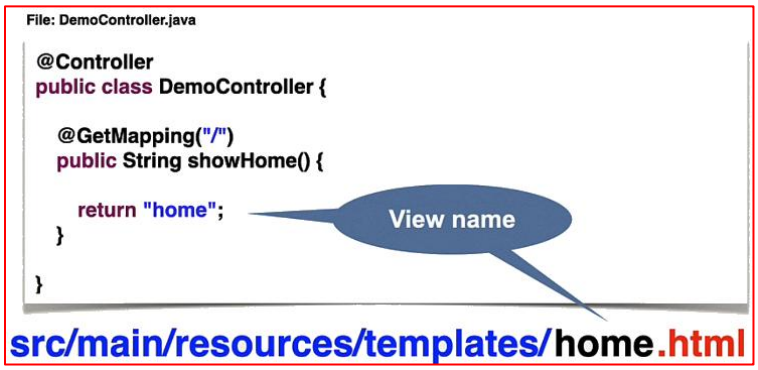
First, we'll create a project on the Spring Initializr website. On this site, we'll add the Maven dependencies necessary for Spring MVC Web App, Security, and Thymeleaf. You'll notice that the website itself will automatically insert this information into our `pom.xml` file. This process will incorporate support for:

- Spring MVC web
- Thymeleaf view
- Spring Security
- Thymeleaf extras, which enable integration between Thymeleaf and Spring Security. (We'll explore examples of this integration in subsequent lessons.)



Step 2: Developing the Spring Controller

Next, we'll develop our Spring Controller. We'll create a standard controller called `DemoController` and set up a `GetMapping` named `showHome`. Our method will return a String, "home," corresponding to our template found at `src/main/resources/templates/home.html`.



Step 3: Developing the Thymeleaf View Page

In this step, we will craft a basic HTML page at `src/main/resources/templates/home.html`. This will serve as the welcome page for the "apple" company, with a simple greeting.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Apple Company Home Page</title>
</head>

<body>

    <h2>Apple Company Home Page - Yoohoo - Silly Goose - NO SOUP!!!</h2>
    <hr>
    Welcome to the Apple company home page!

</body>
</html>
```

Now, let's dive into the details of each step.

Creating the Project (Detailed Instructions)

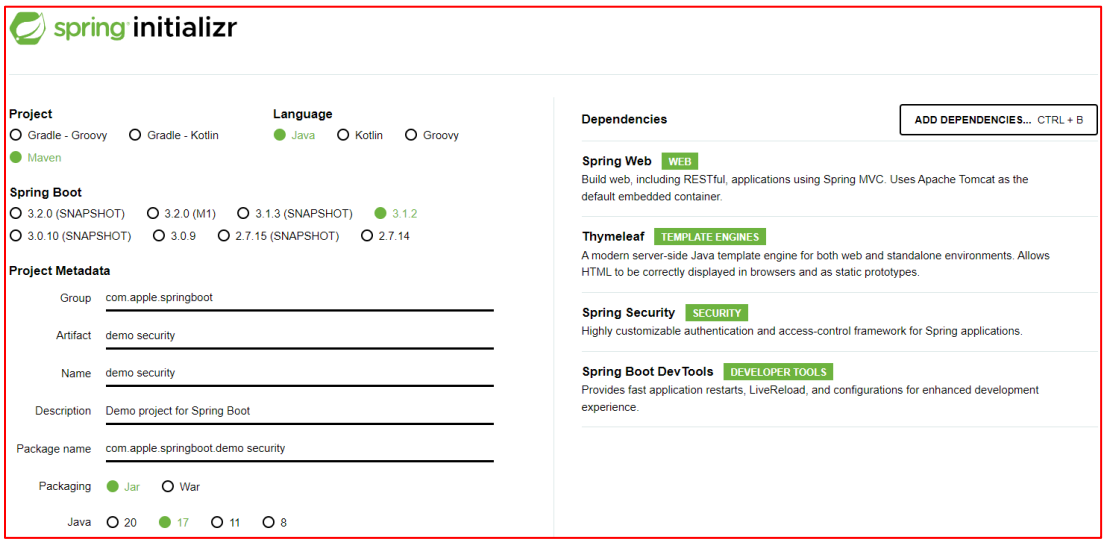
Open your web browser and navigate to start.spring.io. For the project settings:

- Choose "Maven Project."
- Select the language as "Java."
- Pick the most recent version of Spring Boot (note that your version might slightly differ).
- For the group, name it `com.apple.springboot`, and for the artifact ID, give it `demo security`.
- Choose "Jar" for packaging and select the appropriate version of Java.

In the top-right corner, click on "Add Dependencies" and add the following:

- Web
- Thymeleaf
- Spring Security
- DevTools

Make sure these four dependencies are selected, then proceed to generate the project.



After downloading the project, unzip it and reposition the files according to your preferred directory structure. We'll name the folder `08-spring-boot-spring-mvc-security`, moving and renaming the project to `01-spring-boot-spring-mvc-security-default`. You can then open this new Spring MVC Security project in your chosen IDE.

Examining the `pom.xml` File

Let's take a look at the `pom.xml` file, where you'll notice entries for:

- Spring Security support
- Thymeleaf view support
- Spring MVC web support
- Thymeleaf Security support (for integration with Spring Security)

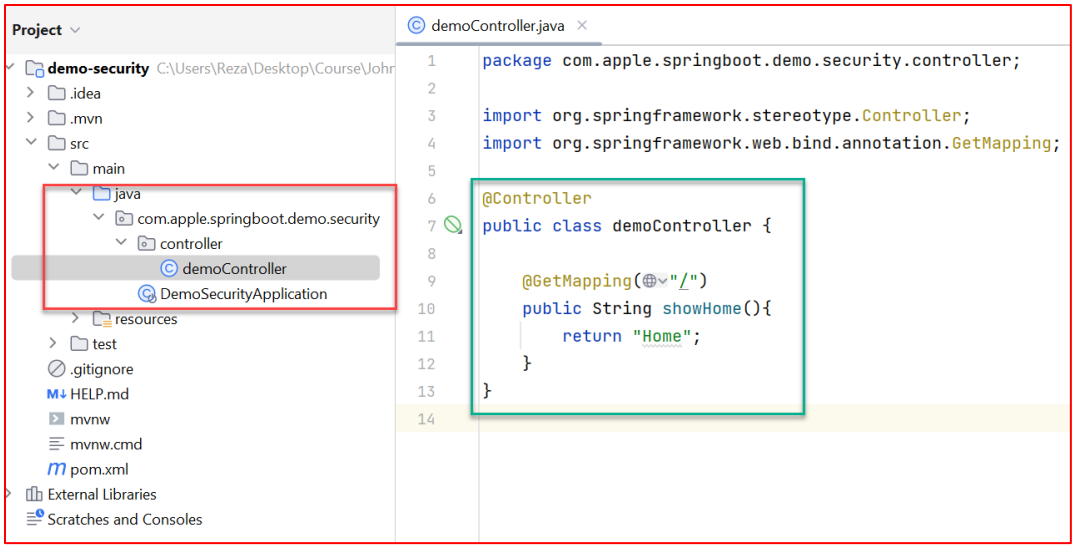
This forms a solid starting point for our project.

Housekeeping and Controller Development

Now, we'll handle some routine housekeeping for setting up the compile options for DevTools. You may follow along, as we've encountered much of this work previously.

In step two, we'll develop our Spring Controller. Create a new package named "Controller" and a new class called `DemoController`. This class will have a `controller` annotation, a `GetMapping`

for the home page, and a straightforward method named `showHome`, returning the view name "home."



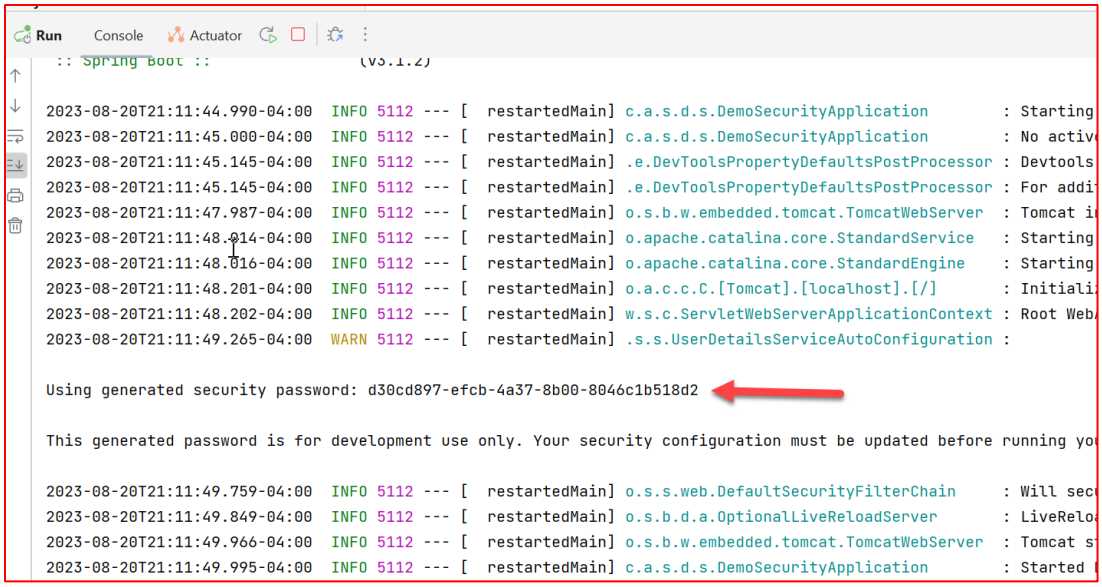
Defining the Thymeleaf View Page

Finally, in step three, we'll define the Thymeleaf view page, creating a new HTML file called `home.html`. Here, you can add a title and body content with an "h2" heading, a horizontal line, and a welcome message to the "apple" company home page.



Testing the Application

We can now test the application, expecting security to kick in due to our `pom.xml` file's configuration. Start the application, and in your browser, go to `localhost:8080`. You'll notice that Spring Security prompts for a user ID and password.



Use the default username, "user," and the generated password from your console. Upon signing in, you should see your crafted page, confirming that Spring Security is working.

